

Cahier des charges — MediPlan

Plateforme web de gestion des rendez-vous médicaux

Cadre	Projet intégrateur 030747 — Collège La Cité
Session	Printemps 2026
Version	2.0 — version finale
Date	12 août 2026
Remplace	v1.0 du 28 mai 2026 (conservée)
Équipe	Souleymane DIALLO · Zakaria Lahouiri · Larbi Saib

Note de révision

La v1.0 annonçait qu'elle serait révisée si le périmètre évoluait. Il a évolué. Cette version dit ce que nous avons livré et où nous nous sommes écartés du plan. La v1.0 n'est pas modifiée : elle reste la trace de ce que nous avons prévu.

Le document est **plus court que la v1.0** : la rétroaction de l'ébauche nous demandait un cahier des charges plus direct. Nous avons retiré les développements qui n'ajoutaient rien et gardé ce qui informe.

Réponses à la rétroaction de l'ébauche #1

Remarque	Ce que nous avons fait
Identifier les fonctionnalités livrées en version minimale	§ 1.3 — le noyau minimal est nommé, et il est livré
Notifications, statistiques, flux du jour, rappels : peut-être de trop	Trois sont livrés, les rappels non. § 2.2
Une seule clinique ou vrai multi-cliniques ?	§ 1.3 — une clinique fonctionnelle . Le multi-cliniques est dans le modèle, pas dans l'interface
Rappels par courriel : prévus ou amélioration ?	Amélioration possible seulement . Reclassés hors périmètre, § 2.2
Vérifier que wireframes, UML et schéma d'architecture sont dans la remise	§ 4.2 — vérifié, avec un manque assumé sur les wireframes
Être clair sur la répartition du travail	§ 4.3, nouvelle section

Sections trop détaillées	Document ramené de 13 à environ 7 pages
--------------------------	---

Les six écarts avec la v1.0

v1.0 prévoyait	La réalité
Déploiement local seulement	Application en ligne sur Azure, infrastructure décrite en Bicep, ~0 \$/mois
Le patient réserve, comme sur Doctolib	La réception réserve pour lui et il peut réserver seul
Disponibilités récurrentes	Plages datées , plus les congés
Modification d'un rendez-vous	Réservation et annulation seulement
Gestion des médecins et des cliniques par l'interface	Non livrée
Tests E2E, tests de charge, audits Lighthouse	203 tests unitaires. Rien d'automatisé de bout en bout

Les deux premiers sont des dépassements, les quatre autres des renoncements.

1. Présentation du projet

1.1 Contexte

Dans beaucoup de cliniques de proximité, les rendez-vous sont gérés au téléphone, sur un agenda papier ou dans un fichier partagé. Il en résulte des doubles réservations, des créneaux perdus quand un patient annule, aucune vue commune sur la journée en cours, et aucune donnée pour piloter l'activité.

Une précision est apparue en cours de projet. La v1.0 plaçait le patient au centre. En travaillant le besoin, nous avons retenu que **dans une petite clinique, le patient appelle — il ne s'inscrit pas.** L'utilisateur qui vit le problème est la réception. Ce déplacement a réorganisé le reste du projet.

1.2 Objectifs

ID	Objectif	Résultat
OM-01	Réserver, modifier ou annuler sans téléphoner	Réservation et annulation livrées ; modification non livrée
OM-02	Le médecin voit sa journée en une vue	Le flux du jour
OM-03	Réduire la charge administrative	Une plage saisie une fois génère ses créneaux

OM-04	Empêcher les doubles réservations	Garantie posée dans la base , pas dans le code
OM-05	Séparer les données par clinique	Chaque requête est bornée par le rôle de l'appelant
OM-06	Sécuriser les accès selon le rôle	RBAC à 4 rôles
OM-07	Fournir des statistiques d'activité	Volumes, absences, occupation

Six objectifs sur sept atteints.

Les objectifs pédagogiques (architecture client-serveur, qualité de code, sécurité, base relationnelle, conteneurisation) sont atteints, avec deux réserves : la dette de formatage n'a pas été résorbée, et la sécurité est incomplète sur les couches secondaires (§ 3.3).

1.3 Périmètre du projet

Le noyau minimal — livré en entier

La rétroaction demandait d'identifier ce qui serait livré coûte que coûte. Voici ce noyau, tel que nous l'avons arrêté, et il est **entièrement livré** :

1. Se connecter, avec des droits qui dépendent du rôle ;
2. Publier une plage de disponibilité, et obtenir ses créneaux automatiquement ;
3. Réserver un créneau pour un patient, sans qu'une double réservation soit possible ;
4. Suivre le rendez-vous dans la journée : arrivé, en consultation, terminé ;
5. Annuler, et récupérer le créneau.

Tout le reste est venu par-dessus.

Livré en plus du noyau

Notifications internes, statistiques et tableaux de bord, export CSV, réservation en libre-service par le patient, mode sombre, mise en ligne sur Azure.

La rétroaction signalait que les notifications, les statistiques et le flux du jour pourraient être de trop. Les trois sont livrés. Les rappels par courriel, quatrième élément signalé, ne le sont pas — et nous les avons retirés du périmètre plutôt que de les promettre.

Une clinique, pas un réseau de cliniques

Question posée dans la rétroaction, tranchée ici : MediPlan vise une seule clinique fonctionnelle.

Le multi-cliniques existe dans le modèle de données et dans la sécurité — chaque compte porte une clinique, chaque requête est filtrée dessus, et un annuaire public liste les cliniques à l'inscription.

Mais **aucun écran ne permet de créer ou de configurer une clinique**. Ouvrir une seconde clinique demande aujourd'hui une intervention en base.

C'était l'ambiguïté principale de la v1.0. Elle est levée.

Non livré, bien que prévu

- gestion des médecins et des spécialités par l'interface *(l'accès rapide est visible mais grisé, plutôt que masqué)* ;
- configuration d'une clinique par l'interface ;
- modification ou déplacement d'un rendez-vous ;
- décalage en bloc des rendez-vous d'un médecin *(codé, testé, non intégré)* ;
- disponibilités récurrentes *(remplacées par des plages datées)* ;
- **rappels par courriel — reclassés en amélioration possible, § 2.2 ;**
- annulation par le patient *(elle suppose une règle de délai minimum que nous n'avons pas écrite ; nous préférons ne pas livrer l'une sans l'autre)*.

Exclu dès l'origine

Dossier médical, prescriptions, paiement en ligne, application mobile native, synchronisation d'agenda externe, IA médicale, intégration hospitalière.

2. Description fonctionnelle

2.1 Besoins et exigences métiers

Problématique

Centraliser les rendez-vous et le flux quotidien d'une clinique de petite taille : supprimer les doubles réservations, réduire le temps passé au téléphone, et ne perdre aucun créneau libéré par une annulation.

Utilisateurs cibles

Rôle	Ce qu'il fait dans MediPlan
Administrateur de clinique *(la réception)*	L'utilisateur principal. Plages, réservations, flux du jour, annulations, statistiques, comptes
Médecin	Consulte sa journée, publie ses disponibilités, fait avancer les statuts, reçoit les notifications
Patient	Existe sans compte (*patient léger*, créé au comptoir), ou réserve lui-même et consulte ses rendez-vous
Super administrateur	Rôle appliqué côté sécurité, sans écran dédié

Scénarios d'utilisation

Les diagrammes de cas d'utilisation sont dans [docs/conception/cas-utilisation/](../conception/cas-utilisation/) : une vue d'ensemble et six diagrammes détaillés.

SC-01 — Réserver. Deux canaux. *À la réception* : médecin, créneau, patient — le patient est créé au comptoir sans compte. *En libre-service* : le patient inscrit choisit un médecin, un créneau et un motif. Les deux passent par la même transaction et le même index en base, donc la garantie ne dépend pas du canal. Seuls les créneaux réellement libres sont proposés. *Écart* : pas de recherche par spécialité, on choisit un médecin.*

SC-02 — Annuler. Le motif est obligatoire : tant qu'il est vide, la confirmation reste inactive. Le créneau redevient reservable immédiatement. *Écarts* : pas de modification ni de replanification, pas de délai minimum, et le patient ne peut pas annuler lui-même.*

SC-03 — Suivre la journée. Vue partagée entre la réception et le médecin. Chaque rendez-vous suit : Réservé Arrivé En consultation Terminé, ou Annulé. Les statuts terminaux demandent une confirmation. Une notification interne est émise à chaque changement. *Écart* : le décalage en bloc en cas de retard n'est pas intégré.*

SC-04 — Configurer une clinique. *Non livré par l'interface* (§ 1.3).

SC-05 — Consulter les statistiques. Volume de rendez-vous, taux d'absence, taux d'occupation, détail par médecin, filtrables par période. Calculés en base, bornés à la clinique. *Écart* : les motifs d'annulation ne sont pas agrégés.*

2.2 Fonctionnalités principales

ID	Fonctionnalité	Priorité v1.0	État
EF-01	Authentification et comptes	Must	Livré
EF-02	Gestion des cliniques	Must	En base et à l'inscription ; pas d'écran
EF-03	Gestion des médecins et spécialités	Must	Non livré
EF-04	Gestion des disponibilités	Must	Plages datées et congés ; pas de récurrence
EF-05	Réservation, modification, annulation	Must	Réservation et annulation livrées ; modification non livrée
EF-06	Flux clinique du jour	Must	Livré
EF-07	Notifications internes	Must	Livré
EF-08	Tableaux de bord et statistiques	Should	Livré

EF-09	Rôles et permissions (RBAC)	Must	Livré
EF-10	Export CSV	Could	Livré
EF-11	Rappels par courriel	~~Should~~	Retiré du périmètre — amélioration possible

Sur 8 exigences *Must* : 4 livrées, 3 partielles, 1 absente.

EF-11, rappels par courriel. La rétroaction demandait si ces rappels étaient réellement prévus.
Réponse : non. Aucune notification ne sort de l'application. Ils passent en amélioration possible (§ 6).
Nous préférons le dire que le laisser croire.

Un point que nous devons signaler. L'export CSV était classé *Could* et il est livré ; la gestion des médecins était classée *Must* et elle ne l'est pas. L'explication est simple : l'export était court et bien défini, la gestion des médecins était longue et floue. Nous avons suivi le plus facile au lieu du plus important, et nous n'avons pas relu notre priorisation en cours de route.

2.3 Interface utilisateur

Exigence v1.0	Résultat
Moins de trois clics pour réserver	Deux clics jusqu'au formulaire
Navigation adaptée au rôle	Le menu change visiblement selon le rôle
Responsive à partir de 360 px	Vérifié
WCAG 2.1 AA	Travaillé, non certifié — 9 avertissements d'accessibilité restent

Ajouts non prévus : un design system tokenisé ([docs/frontend/design-system.md](../frontend/design-system.md)), une refonte complète de l'interface, et un mode sombre.

Wireframes. Ils n'ont pas été produits en amont. Nous avons conçu l'interface en la codant, puis documenté le résultat. Ce qui en tient lieu aujourd'hui : les écrans réels, tous capturés et commentés dans le [manuel d'utilisation](../guide-utilisation/README.md). C'est un manque assumé, pas un oubli déguisé.

2.4 Conditions d'utilisation

Environnements. Ordinateur (Windows, macOS, Linux), tablette à partir de 768 px, téléphone à partir de 360 px via navigateur. Aucune application native. L'application s'utilise sans rien installer.

Navigateurs. Développé et vérifié sur **Chrome et Edge**. Firefox et Safari étaient annoncés dans la v1.0 mais **n'ont pas été testés**.

Limites d'usage.

- connexion Internet requise, aucun mode hors ligne ;
- démarrage à froid de 10 à 15 secondes après une période d'inactivité —

contrepartie du **scale-to-zero** qui maintient le coût à ~0 \$/mois ;

- session de 60 minutes, sans rafraîchissement automatique ;
- verrouillage du compte après cinq échecs de connexion ;
- pas de pagination serveur : le volume académique le permet, une clinique à fort

volume non ;

- données de démonstration entièrement fictives. Aucune donnée réelle de patient n'a été manipulée, et la plateforme n'est pas prête à en recevoir (§ 3.3).

3. Description technique

3.1 Technologies à utiliser

Couche	v1.0 annonçait	Utilisé
Frontend	Angular + Material	Angular 22 standalone, Material 3, Tailwind 4
Backend	NestJS	NestJS 11, API préfixée <code>api/v1</code>
ORM	TypeORM ou Prisma	TypeORM, migrations versionnées
Base	PostgreSQL	PostgreSQL, infogéré chez Neon
Tests	Jest + Cypress/Playwright	Jest seul
Conteneurs	Docker Compose	Docker Compose + Azure Container Apps
Suivi	GitHub Projects	Jira

Quatre outils annoncés n'ont pas été installés : Helmet, Swagger, Cypress et Artillery/k6. Les conséquences sont aux § 3.3, 3.4 et 4.2.

3.2 Architecture du système

Architecture client-serveur en trois tiers. En développement, les trois services tournent sous Docker Compose. En ligne :

```

Navigateur
| HTTPS
Frontend Angular servi par nginx -- Azure Container Apps, ingress public
| proxy /api/
API REST NestJS -- Azure Container Apps, ingress INTERNE
  
```

Trois décisions n'étaient pas dans la v1.0 :

6. **Le backend n'a aucune adresse publique.** Il n'est joignable que par le frontend. Cela le met hors d'atteinte depuis Internet et supprime tout besoin de CORS, puisqu'il n'y a jamais de requête inter-origines.

7. **L'infrastructure est décrite en Bicep** ([infra/](../infra/)). Aucune ressource n'a été créée à la main.

8. **Le schéma de base est piloté uniquement par des migrations versionnées.**

Aucune synchronisation automatique : n'importe qui reconstruit le même schéma.

Aucun système externe n'est intégré, conformément au périmètre.

3.3 Sécurité

En place : hachage bcrypt à coût 12 ; verrouillage après cinq échecs ; RBAC à 4 rôles avec gardes sur chaque route protégée ; séparation des données par clinique appliquée côté serveur ; validation stricte des entrées, qui **rejette** tout champ non déclaré au lieu de l'ignorer ; requêtes paramétrées via l'ORM ; HTTPS ; erreurs normalisées sans fuite d'information ; aucun secret dans le dépôt.

Absent, alors que la v1.0 s'y engageait :

Manque	Conséquence
Pas de jeton de rafraîchissement ni de rotation	Un seul JWT de 60 minutes, non révocable avant expiration
Pas de déconnexion côté serveur	Le jeton reste valide jusqu'à son expiration
Aucun en-tête HTTP de sécurité	Ni CSP, ni HSTS, ni X-Frame-Options. Helmet n'est pas installé
Aucune journalisation d'audit	Aucune traçabilité des accès
Aucune limitation de débit	Le verrouillage protège un compte, pas l'API

La protection CSRF est sans objet : l'authentification passe par un en-tête Authorization, pas par un cookie de session.

Le socle tient. Ce qui manque relève de la défense en profondeur, et les en-têtes de sécurité comme la journalisation demandent moins d'une journée. La v1.0 s'engageait sur l'OWASP ASVS niveau 1 ; nous ne pouvons pas prétendre l'avoir atteint, faute de l'avoir vérifié point par point.

3.4 Performance et scalabilité

Performance. Sur les cinq exigences chiffrées de la v1.0, une seule a été vérifiée — mais c'est la seule qui porte une garantie d'intégrité plutôt qu'un confort :

Exigence	État
API sous 300 ms au 95 ^e percentile	Non mesuré. Observé : sonde 0,44 s, connexion 0,99 s
Frontend sous 2 s	Non mesuré. Premier accès 9,9 s, dû au réveil des conteneurs
Tableau de bord de 100 RDV sous 1 s	Non mesuré
50 utilisateurs simultanés	Non testé
Aucune double réservation en accès concurrent	Vérifié en ligne : deux requêtes simultanées, une 201 et une 409

Scalabilité. Le backend est sans état — l'authentification passe par un jeton, aucune session n'est gardée en mémoire — donc plusieurs instances peuvent coexister, et Container Apps sait les répliquer par configuration. L'intégrité ne dépend pas du nombre d'instances, puisque la garantie est dans la base.

Mais rien de tout cela n'a été testé sous charge. Et le premier mur ne serait pas la capacité serveur : c'est l'**absence de pagination**, qui fait charger une liste de rendez-vous en entier.

4. Planification et livrables

4.1 Phases du projet

La v1.0 prévoyait cinq phases successives. Nous avons travaillé en sprints, avec un découpage **par tranche verticale** : chacun mène sa fonctionnalité de la migration jusqu'à l'écran, plutôt qu'une séparation « un front / un back ». Le développement frontend et backend s'est donc fait en parallèle, et la mise en ligne pendant le dernier sprint plutôt qu'à la fin.

Dates relevées dans l'historique du dépôt :

Date	Jalon
28 mai 2026	Cahier des charges v1.0
3 juin 2026	Dossier de conception déposé
17 juin 2026	Socle technique et authentification complets *(PR #1 à #12)*
8 juillet 2026	Patient léger, disponibilités, prise de rendez-vous
21-22 juillet 2026	Refonte UX/UI, KPI réels du tableau de bord
29 juillet 2026	Mise en ligne sur Azure *(PR #16 à #18)*
10 août 2026	Statistiques, export CSV, notifications *(PR #19 à #24)*

11 août 2026	Réservation patient en libre-service *(PR #25 à #31)*
13 août 2026	Présentation finale

Notre rythme a été irrégulier : deux semaines à deux commits, deux autres à trente-neuf. Nous avons produit à l'approche des échéances plutôt que régulièrement, et c'est la même cause qui explique le problème d'intégration décrit au § 4.3.

4.2 Livrables attendus

ID	Livable	État
LIV-01	Code frontend + tests	apps/frontend — 133 tests
LIV-02	Code backend + tests	apps/backend — 70 tests
LIV-03	Schéma versionné par migrations	7 migrations
LIV-04	Documentation technique	README, CONTRIBUTING, docs/
LIV-05	Cahier des charges final	ce document
LIV-06	Diagrammes UML et ERD	[docs/conception/](../conception/) — 7 cas d'utilisation, classes, 3 séquences, ERD, avec une explication écrite chacun
LIV-07	Wireframes	Non produits. Remplacés par le design system et le manuel d'utilisation (§ 2.3)
LIV-08	Application démontrable	En ligne, plus une vidéo de 4 min
LIV-09	Docker Compose et Dockerfiles	Démarrage en une commande
LIV-10	Documentation OpenAPI	Non produite — Swagger n'est pas installé
LIV-11	Support de présentation	16 diapositives

Vérification demandée par la rétroaction — les annexes sont-elles présentes ?

- **Diagrammes UML** : oui, 11 diagrammes dans docs/conception/, en Mermaid et exportés en images.
- **Schéma d'architecture** : oui, § 3.2 de ce document, et détaillé dans [infra/README.md](../infra/README.md).
- **Wireframes** : **non**, et c'est le seul manque. Voir § 2.3.

Produits en plus : infrastructure Azure en Bicep, rapport final de projet, manuel d'utilisation, document de tests et résultats, contributions individuelles, vidéo de démonstration.

4.3 Répartition du travail

Section ajoutée à la demande de la rétroaction.

Le découpage est par domaine fonctionnel, chacun menant sa partie de la base de données jusqu'à l'écran.

	Souleymane DIALLO	Zakaria Lahouiri	Larbi Saib
Domaine	Socle, sécurité, mise en ligne, espace patient	Le temps du médecin	Le rendez-vous et sa mesure
Conception	Cahier des charges, 7 cas d'utilisation, ERD	Diagramme de classes	Diagrammes de séquence
Développement	Monorepo, Docker, CI, authentification, JWT, RBAC, design system, refonte UI, index unique partiel, annulation, réservation patient, déploiement Azure	Disponibilités et génération des créneaux, congés, flux du jour, notifications internes, export CSV	Patient léger, socle du rendez-vous, contrainte anti-double-réservation, statistiques et tableaux de bord
Épisodes Jira	E1, E2, E4, E7	E3, E5, E6	—
Commits sur `main`	72	8	4

Le volume de commits est très inégal et nous l'assumons : Souleymane a porté le socle et l'intégration, ce qui produit mécaniquement beaucoup de commits, tandis que Zakaria et Larbi ont livré des fonctionnalités complètes en peu de commits. Ce chiffre mesure une façon de travailler, pas une contribution.

Le détail par personne — réalisations, difficulté rencontrée, solution apportée — est dans [docs/presentation/CONTRIBUTIONS.md](../presentation/CONTRIBUTIONS.md), retracé commit par commit.

Le problème d'intégration. Des branches sont restées non fusionnées pendant des semaines, jusqu'à 56 commits de retard sur main. Cinq tickets étaient marqués « Terminé » sans exister dans le produit. Nous avons réintégré trois branches par pull requests revues, repassé les autres en « À faire » avec la raison écrite, et resserré notre définition de « terminé » : **terminé = fusionné dans `main`, CI verte.**

5. Modalité de validation

Ce que nous avons testé

Nous n'avons pas visé une couverture uniforme. Nous avons testé **ce qui casse sans prévenir** : les règles métier, la sécurité, et le comportement des formulaires. L'apparence est hors périmètre — un test qui vérifie une couleur casse à chaque retouche et n'attrape aucun défaut réel.

203 tests automatisés, tous verts au 12 août 2026 : 70 côté backend, 133 côté frontend. Détail dans [docs/tests/plan-et-resultats.md](../tests/plan-et-resultats.md).

Validation manuelle. Le parcours complet est rejoué avant chaque démonstration, selon un scénario écrit en le jouant sur l'application en ligne. Le passage du 10 août a vérifié 14 points, sans aucune erreur dans la console du navigateur.

Test de concurrence. Deux réservations simultanées sur le même créneau, sur l'application déployée : une requête en 201, l'autre en 409.

Ce que nous n'avons pas testé

Aucun test de bout en bout automatisé. Aucun test de charge. Aucun audit Lighthouse. Aucun rapport de couverture, alors que la v1.0 visait 70 % sur les modules métier. Aucun critère d'acceptation au format Gherkin, aucune enquête de satisfaction.

Indicateurs

Les sept indicateurs chiffrés de la v1.0 supposaient un outillage que nous n'avons pas installé. **Aucun n'a été mesuré selon la méthode annoncée.** Plutôt que de les maquiller, voici ce qui a réellement été vérifié le 12 août 2026 :

Indicateur	Résultat
Tests automatisés verts	203 / 203
Double réservation en accès concurrent	Impossible — vérifié en ligne
Application en ligne	Sonde 200 en 0,44 s ; connexion 200 en 0,99 s
Parcours complet rejoué	14 points vérifiés
Erreurs dans la console	Aucune
Fonctionnalités livrées	9
Coût d'hébergement	~0 \$/mois

Définition of Done

La v1.0 en proposait une de six critères. Elle n'a pas tenu : trop longue à vérifier, personne ne la relisait. Elle a été remplacée par une règle que nous avons réellement appliquée — **terminé = fusionné dans `main`, CI verte** — parce que c'est celle que l'outillage vérifie tout seul.

Une définition de « terminé » que rien ne vérifie n'est pas une définition.

6. Conclusion

MediPlan est livré : une application déployée, testée, et démontrable de bout en bout. La double réservation est devenue techniquement impossible, la journée de clinique tient dans un écran partagé, et un créneau annulé repart à la réservation.

Le produit est en deçà du plan initial sur trois points : la gestion des médecins et des cliniques par l'interface, la modification d'un rendez-vous, et la validation automatisée de bout en bout. Il le dépasse sur deux autres : la mise en ligne réelle, jamais exigée, et le second canal de réservation.

La cause des manques n'est pas technique, elle est de méthode. Nous avons travaillé longtemps sans intégrer, et notre définition de « terminé » a laissé du code exister sans être dans le produit. Le correctif est appliqué, et c'est probablement ce que nous retenons le plus de ce projet.

Ce que nous ferions ensuite

D'abord finir ce qui est écrit : intégrer le décalage en bloc, livrer la configuration de clinique et la gestion des médecins, passer le formateur sur tout le dépôt.

Puis fiabiliser : poser les en-têtes de sécurité et la journalisation d'audit (moins d'une journée pour notre plus grand écart), ajouter un jeton de rafraîchissement, écrire les tests de bout en bout, ajouter la pagination serveur.

Ensuite seulement, étendre : les **rappels par courriel ou SMS** en premier — c'est la fonctionnalité qui attaque directement le taux d'absence, le seul chiffre que la clinique voit sur sa facture. Puis une liste d'attente qui proposerait automatiquement un créneau libéré. Puis l'annulation par le patient avec son délai minimum. Et enfin l'ouverture à un réseau de cliniques, qui demanderait de construire l'interface d'administration aujourd'hui absente.

L'ordre n'est pas arbitraire : ajouter sur une base fragile est ce qui nous a coûté le plus cher.

Documents liés

Document	Contenu
[Rapport final](../RAPPORT-FINAL.md)	Comment le projet a été conduit, et ce qui a changé
[Dossier de conception](../conception/)	Les 11 diagrammes et leurs explications
[Manuel d'utilisation](../guide-utilisation/README.md)	L'application écran par écran
[Tests et résultats](../tests/plan-et-resultats.md)	Stratégie, résultats, ce qui n'est pas couvert
[Contributions individuelles](../presentation/CONTRIBUTIONS.md)	Qui a fait quoi, commit par commit
[Déploiement Azure](../deployment/azure.md)	La mise en ligne